ECE 350 Final Project - Technical Report
Andrew McKnight and Claire Andreassen
April 22, 2025

Project Overview

This project implements a **mini digital drum set** using an FPGA programmed in Verilog. The design is inspired by electric drum kits, which provide a compact and quiet alternative to traditional acoustic sets. Each drum pad – seven in total – features a force-sensitive resistor (FSR) that detects hits and sends analog signals to the FPGA via voltage divider circuits. The system includes a custom processor, developed throughout the semester, which interprets these signals and triggers sound playback using memory-mapped I/O. Drum sounds are stored in external memory modules and played back through pulse-width modulation (PWM), which approximates the audio waveform for each hit.

Figure 1 - finished product, desktop drum kit with 7 different drum sounds

Sound Design and Playback

We sourced drum sounds from YouTube and edited them in Audacity to have a 24 kHz sample rate with 16-bit signed samples, a combination that we found maximized audio quality while allowing all the files to fit in the FPGA's onboard memory. For reference, CDs typically have 44.1 kHz sample rates and 16-bit sample depths. With this in mind, our sound quality was relatively high given the limitations of FPGA audio playback (specifically, the limited storage availability).

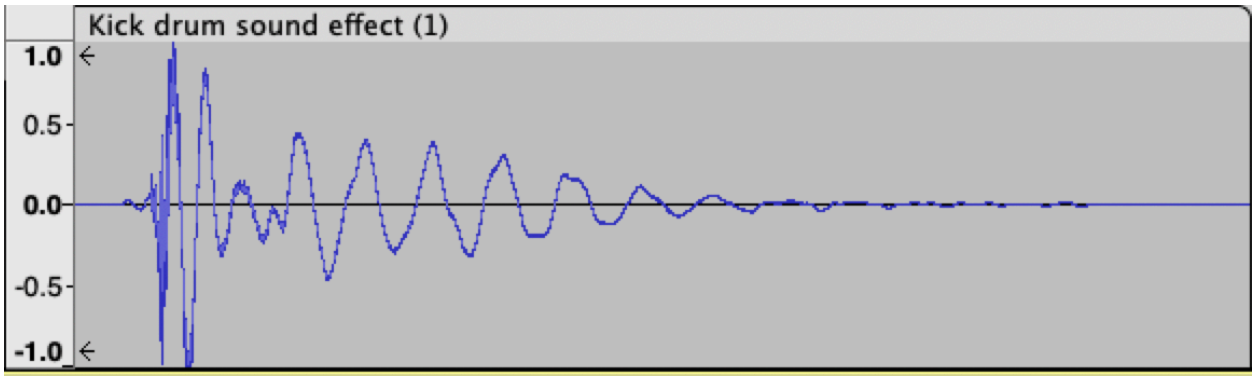

Figure 2 - Finalized kick drum sound viewed in Audacity

Each sound file was scaled in Audacity to maximize volume without causing distortion, and a fade-out effect was applied to prevent clipping at the end of the sound caused by a large jump in amplitude. The length of each file was trimmed to be as short as possible while including enough of the drum's reverb for the sake of realism. After exporting these sounds from Audacity as .wav files, they were converted to .mem files using a python script, wav_to_mem.py, that ignored .wav file headers and just included the samples themselves. Since the samples were signed, and our serializer required unsigned values, the program added 32768 (which is 2^{15}) to each of the samples, so that they were all positive. This had the implication that they were no longer centered around 0, but rather 32768, which became our "relative 0."

Since the system clock operated at 50 MHz, we needed to implement a clock divider that would generate a second clock at our sample rate of 24 kHz. To do this, we had an always @(posedge clock) block that would increment a counter register, and when the counter reached a counter_limit, would reset the counter to 0 and toggle the sample clock. This toggling needed to happen twice within each period of the sample clock, so we used this expression to calculate counter_limit: sys_freq / (2*sample_freq) - 1.

Also, there was a disparity between the 16-bit sample depth and the processor's 32-bit wordsize. Instead of padding the upper 16 bits of each data entry with 0's, which would have been easier but would have wasted half of the memory, we opted to store two samples in each data entry. This meant that we needed to include a buffer that specified which sample to use: sample "0" or "1" (most-significant and least-significant 16 bits, respectively). This buffer was toggled at the sampling rate, and whenever it was equal to 1 at (posedge sample_clk), the address would be incremented by 1. This address was used to index the memory blocks, which were pre-loaded with sound data using the $readmemh function in Verilog.

Once the samples were read, they were fed into a 16-bit PWM serializer module. This module converted each sample into a pulse-width modulated signal, where the duty cycle (the proportion of time the signal stays high within a fixed period) corresponded to the sample's amplitude. By varying the duty cycle over time, the PWM output approximated the original analog waveform. This output was wired to the FPGA's audio jack. The overall structure used to achieve audio playback is shown in figure 3.

Sensor Configuration

The sensors, which were used to detect drum hits, needed to be highly responsive. When a drum is hit, the drumstick and drum often make contact for just a split-second. Also, there needed to be a broad enough area on the physical drums where a hit could be detected. With these constraints in mind, we opted to use force-sensitive resistors. These work by changing their resistance based on the pressure applied to them. Inside, there is a conductive material that lowers its resistance when compressed. When no force is applied, the resistance is very high – effectively an open circuit. When force is applied, the resistance drops down to a few hundred ohms. We found that this behavior could be utilized with a voltage divider circuit where nearly all the voltage drop would be across the sensor when it is not pressed, versus close to none of the voltage drop when the sensor is being pressed. With the right V_{CC} and resistance values, this voltage drop could be interpreted as a logical 0 or 1. Thus, we could feed a wire directly from the circuit into the JD pmod ports and access these inputs in our wrapper module. The FSRs used are shown in figure 4, and figure 5 includes the sensor circuits and their connection to the FPGA.

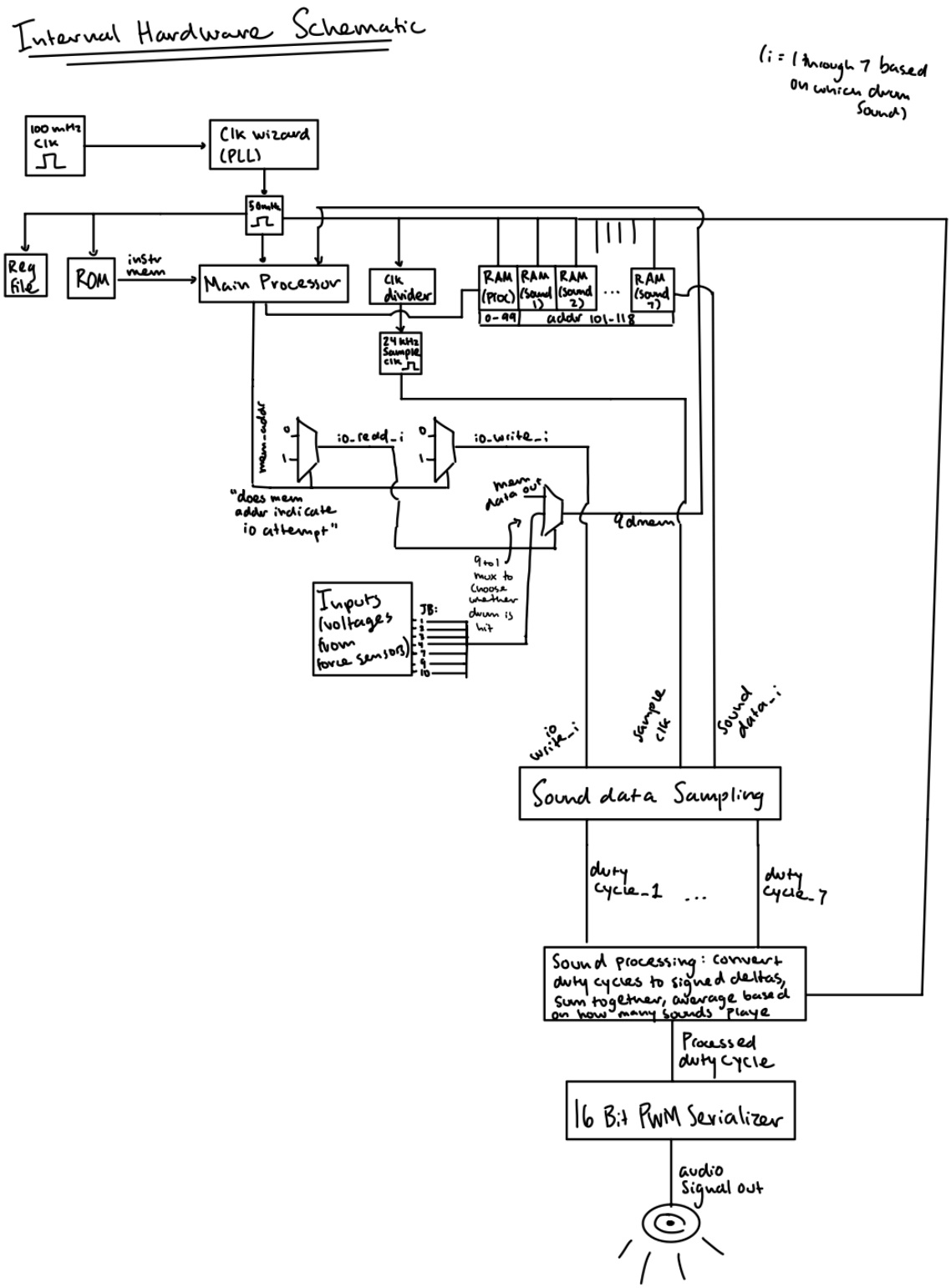

Figure 3 - Internal hardware schematic diagram

Figure 4 - Two FSR models used - 2” (left), 0.5” (right)

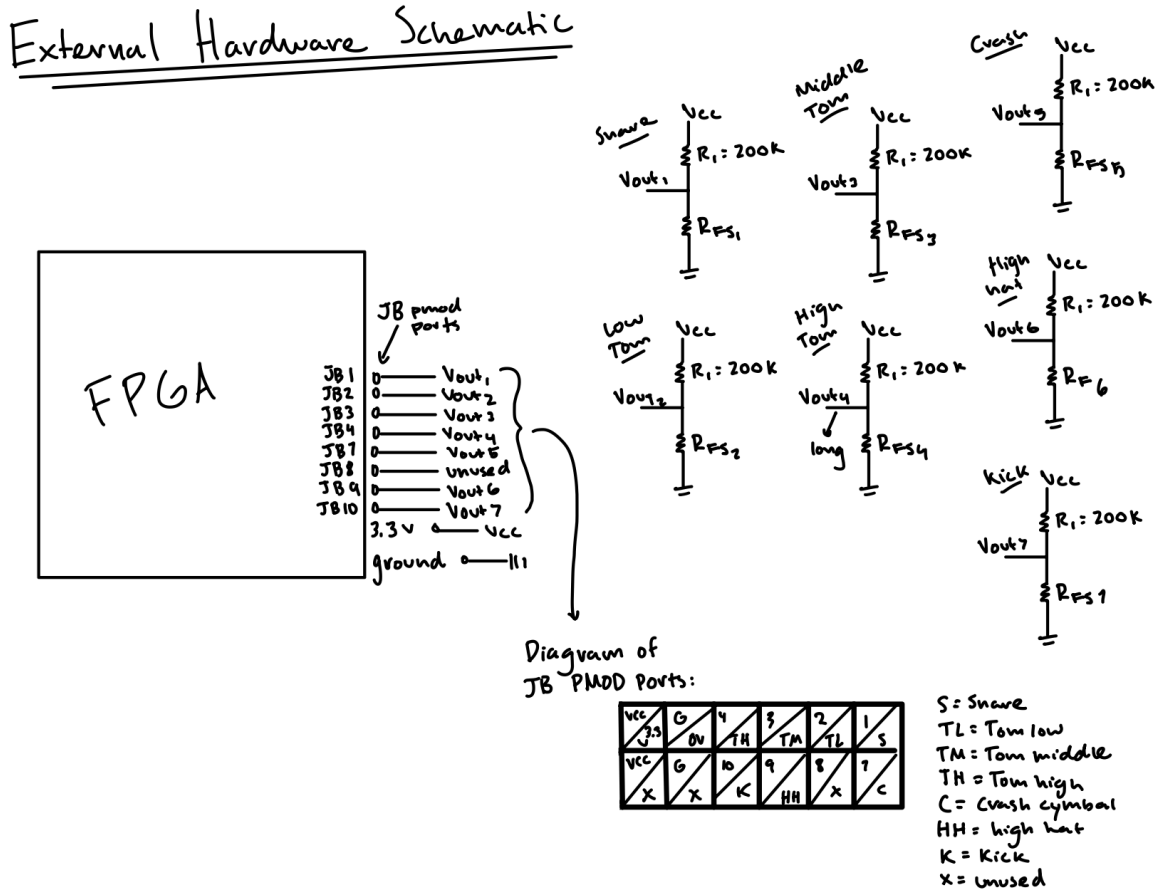

Figure 5 - External hardware schematic diagram

The Processor: Usage and Programming

The processor's main functionality is to probe the FSR inputs and begin playback of the appropriate audio file when a drum hit is detected. To accomplish this, we used a memory-mapped I/O scheme. Inputs – the states of the sensors, represented as 1s and 0s – are read by the processor using lw instructions from designated addresses. Addresses 101 through 108 represent sensors 1-8, respectively. The wrapper module has a series of muxes that control the data sent to the processor. If it sees the processor is attempting to access addresses 101-108, it will send the corresponding sensor data rather than the actual RAM data.

Outputs, from the processor's perspective, are signals that start audio playback in the wrapper module. Using sw instructions with addresses 111-118, the processor is able to communicate which sounds to play. Once the wrapper detects an I/O write corresponding to the ith sound, it writes a 1 to a register called start_soundi using an always @(posedge io_writei) block. Then, it plays the sound in its entirety.

The assembly code's structure is to iterate through the different sensors, reading their state and performing an I/O write if applicable. It continuously repeats this procedure, akin to a "while true" loop, by jumping back to main at the end of the program. Each sensor check calls a function using jal that will store the desired sensor value in $v0. Then, a blt is used to skip the function call for playing sound when the sensor value is 0. If the sensor value is 1, this function call will not be skipped, and the CPU will perform an I/O write.

Given the effectiveness of the memory-mapped I/O based approach to sensor probing and beginning audio playback, we did not need to create any custom processor instructions. Also, our processor was able to operate at the 50 MHz clock requirement, verified by timing analysis in Vivado, so we did not need to optimize it further for efficiency.

Physical Construction

To build the desktop drum kit, we designed laser cut pieces in Adobe illustrator such that the largest piece would measure 5.5 inches in diameter. We then cut a "stage" out of foam that we painted with black acrylic and glued the drum kit onto the stage. This served the purpose of stabilizing the kit pieces. We then attached force sensors (that were soldered onto long wires) to each drum. Zip ties and hot glue were used to reduce tension and organize wiring. These wires were connected to a protoboard that contained fixed resistors (for the upper half of the voltage divider). The protoboard took V_{CC} = 3.3V and ground from the FPGA and gave V_{OUT} back to the PMOD digital input ports.

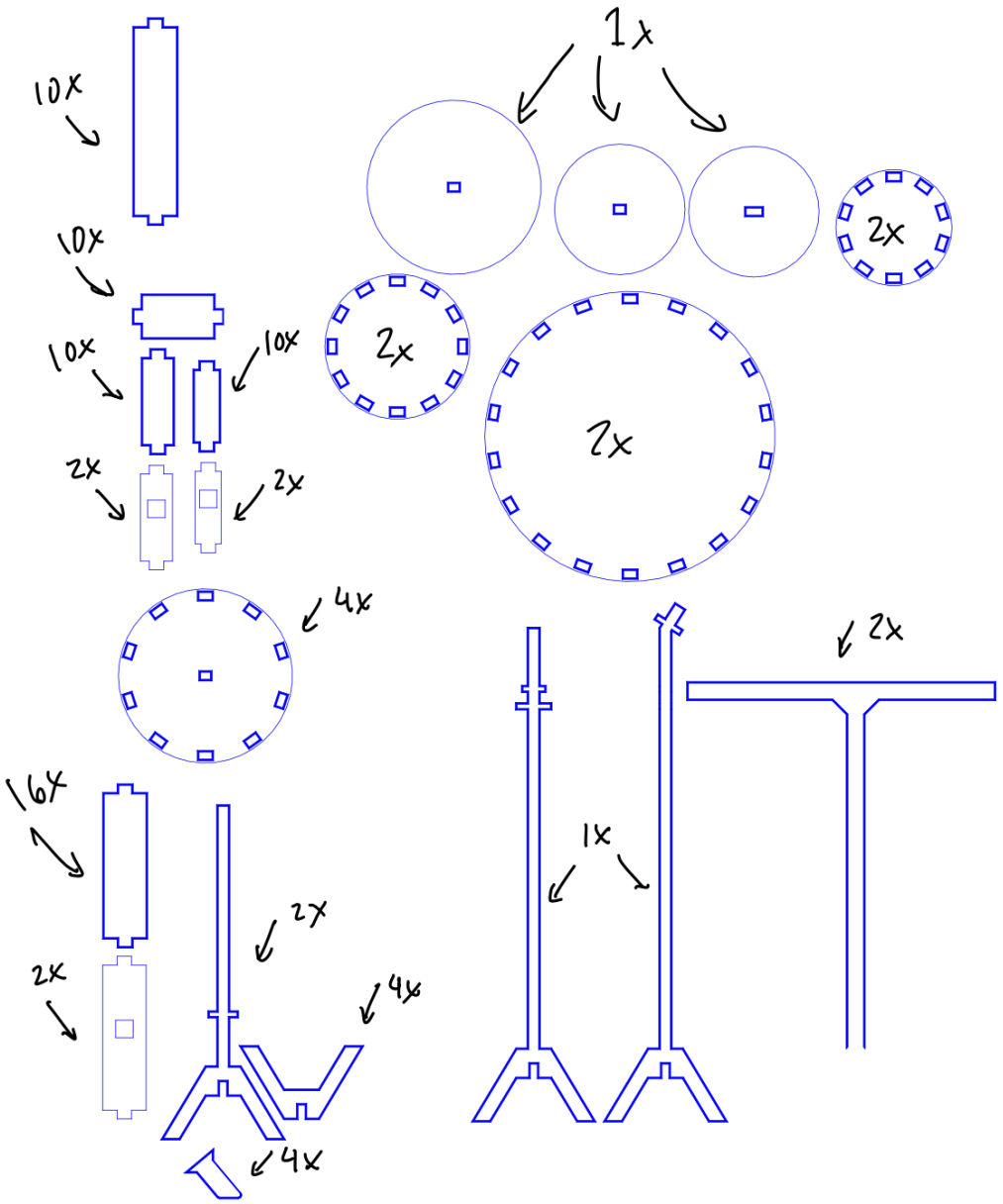

Fig 6 - laser cutting schematic designed with Adobe Illustrate (edited from file downloaded online)

Design Process, Challenges, and Testing

1) One sound on button press

We began our project by trying to get a sound file to play upon the press of a button, referencing Lab 9 code to learn how audio sampling and PWM worked with mem files. This required creating a sampling clock that was the same as the sampling rate we used to export our audio files. This also required creating a mux that would choose whether to play the PWM output or nothing (1'b0) based on whether the button was pressed. We had issues with this step before realizing that a) buttons should be debounced and b) the button press mux works better if placed after the PWM output (as opposed to controlling the input). To test this step, we used a "sweep" sound file that played the range of audible frequencies.

2) Force sensor validation

Meanwhile, we worked on configuring the force sensors such that they could be used to trigger input pins on our FPGA. This involved creating voltage divider circuits with a fixed resistance resistor and the variable resistance force sensors. At first, we wanted to use ADCs to convert the output voltage into an 8 bit value that we would send into 8 different FPGA input pins for each drum. We would then use our processor to determine whether or not the 8 bit value was less than or greater than some threshold, and would have different thresholds for each volume level. This proved challenging and also would only allow us to implement 4 different drums (because there are only 4 PMOD ports, each with 8 digital inputs). We then attempted to use the voltage divider connected to MOSFET transistors connected to a load to generate an output voltage that would be ~3.3 when the force sensor was unpressed and ~0 when pressed. This proved finicky as well. Ultimately, we decided on a much simpler sensor configuration (described earlier), where we increased the fixed resistor resistance (to create a "stronger" voltage divider) and used the 3.3V output on the FPGA as our input voltage.

3) Multiple sounds on button press

We then shifted to using a set of 16 kHz sampled, 8 bit, drum sound files that were closer to what we wanted to use in our final product, and worked on getting multiple different sounds to play on the press of different buttons. We edited the mem files such that there were four 8-bit samples stored at each address – in keeping with our processor's 32 bit word standard. This involved adding together and then averaging duty cycles based on how many buttons were pressed at once**.** One debugging technique we used for this step was assigning the "play_sound" signals to LEDs to see when the module was attempting to play each sound.

4) Single sound on button press using memory-mapped I/O

To make use of our processor, we decided to use memory mapped I/O to read inputs from the FPGA. This involved writing assembly code that looked at specific addresses corresponding to button inputs, checked whether a button was pressed, and, if so, wrote to the corresponding sound-start addresses to tell the wrapper module to play the sound.

5) Multiple sounds with memory mapped io on button press

We extended our single-sound assembly code to handle multiple button reads and multiple sound triggers and also generated more mem files corresponding to different drum sounds. At this point, we switched from using 16 kHz, 8 bit unsigned sound files to using 24 kHz, 16 bit signed sound files to better use the FPGA's RAM capacity, improving sound quality. At first, we concatenated all of these files together and stored them in a single RAM block, keeping track of which address was the "starting address" for each sound. Later, we decided to separate out the sound files into different RAM blocks again, so that each could be simply addressed individually, allowing multiple sounds to be played at once.

6) Sound quality improvements

Sound quality was one of the major challenges throughout the project. When using the 16 kHz, 8 bit sound data, we found that low frequency sounds (e.g. the kick and the low tom) sounded fuzzy and blown out. However, after simply increasing the sample rate and bit depth in Audacity, we found little-to-no improvement in sound quality. Fixing this issue involved changing the PWM serializer to take 16-bit samples and to operate with a longer period that would be able to reflect the more precise samples.

7) Multiple sounds with memory mapped io on force sensor hit

At this point, the only things left to implement were the force sensors. This involved soldering together voltage divider circuits onto protoboards with long extension wires that could reach to each of the force sensors on the drums. There were some small challenges associated with this step – at first we had a few "cold joints" that prevented some parts of the circuit from working, and we also had one force sensor that didn't function. To handle this, we tested the circuit one force sensor at a time using the LEDs on the FPGA. At that point, we assigned each "button_state" variable to the (notted) value on the JB PMOD inputs, which were hooked up to the voltage divider circuits, and we were done!

Potential Improvements

a. Volume control

In the current system, we could achieve volume control via external amplifiers hooked up to the output of the FPGA. The use of a speaker with a volume knob would also work. However, with the aim of mimicking a physical drum kit as closely as possible, we would like to have the volume of the output reflect the strength of the user's hit. To accomplish this, we would have to use ADCs to measure the output voltage of our voltage divider circuit. In our current setup, the voltage divider circuits are configured such that any hit registers as roughly 3.3 V (which the FPGA interprets as a logical 1). If we were to use ADCs, we would have to use a smaller resistance for R_fixed (shown in figure 5). This would allow for a system where the output voltage of the divider circuit V_out = V_in * R_sensor/(R_sensor + R_fixed) doesn't go immediately to zero when R_sensor is reduced, allowing for a wider range of intermediary values that could be measured using an ADC and used to choose between a set of different output volumes. We would modify the memory-mapped I/O system to read 8-bit digital values instead of boolean "pressed" signals and scale combined duty cycles appropriately to create lower average voltages (softer sounds) and higher average voltages (louder sounds).

b. Enable back-to-back drum hits

In the current system, while different drums can be played all at once, the same drum sound cannot be retriggered until the previous playback finishes. This reduces the user's ability to play fast songs with many consecutive (within ~0.4 seconds for most sounds) hits on the same drums. To fix this, we could have a hit-counter variable that would count whether or not the same drum was hit a second or third time while the sampling was still occurring – i.e. if a second or third hit occurred while soundi_addr < sound_file_depth. The incrementation of this variable could then forcibly reset the address at which we are sampling back to zero, causing the sound file to replay from the beginning.